

Senus PLC Board Report — Build Summary

A distillation of the project's commit history and technical decisions, for evaluation. Assiduous Technology Graduate Assessment.

What was built

An AI-native platform that extracts Senus PLC's HY2026 half-year results into a database and serves them as an interactive Board Report, addressing all four readers the brief names — Management, the Board, Equity Investors, and Credit Providers — with a role-scoped view enforced server-side, plus an AI-drafted Directors' Report narrative intended to save real time on public correspondence. FastAPI + SQLAlchemy + Postgres/pgvector backend, Next.js/TypeScript frontend, Claude for extraction and commentary generation, Docker Compose + GitHub Actions CI. 30 backend tests, all passing.

How it was built — commit-by-commit narrative

Phase 0 — Tooling & architecture

- Project tooling (.gitignore, .dockerignore, Makefile) before any application code.
- ADR 0001: LLM extraction over table-parsing, justified by facts (bookings, pipeline value) that only exist in the filing's prose, not its statement tables — plus a reconciliation validator so extracted figures are checked, not trusted.
- ADR 0002: Postgres+pgvector as schema-of-record; SQLite as a zero-setup local fallback; grounded (not vector-search) retrieval for /insights, since only one filing period of data exists.
- ERD for the eight-table data model; the actual HY2026 PR committed as the extraction source.

Phase 1 — Backend core

- SQLAlchemy models with a portable pgvector/JSON embedding column, so the same models run against Postgres or SQLite.
- Extraction pipeline: an LLM path (Claude, strict JSON schema) and a deterministic regex fallback over the same statement sections when no API key is set — both produce the same typed output, both pass through the same reconciliation check.
- Metrics engine: pure functions for Growth, Profitability, Cash & Liquidity, Solvency, and Returns, including an EBITDA→operating cash flow→free cash flow bridge.
- JWT auth, the five category endpoints, /insights (grounded commentary + free-form Q&A).
- Ingestion script tying extraction → reconciliation → metrics → database into one idempotent command.

Phase 2 — Verification

- Unit tests asserting against the filing's own printed figures (e.g. the computed 4.1% revenue growth against the PR's own headline), not synthetic fixtures — plus a deliberately corrupted input proving the reconciliation validator actually catches mismatches.
- Alembic migration mirroring the ORM models for Postgres/pgvector, verified via an offline SQL dry run.

Phase 3 — Frontend

- Next.js App Router shell: auth, typed API client, category page shell reused across all five Board Report sections (KPI tiles, comparison chart, source-evidence table with confidence badges).
- AI Insights page: per-category commentary plus a free-form question box — an interactive surface, not a static export.

Phase 4 — Infrastructure

- Docker Compose (Postgres + API + web) and a GitHub Actions pipeline running the test suite and a Next.js production build on every push.
- Fixed a real Docker bug before it shipped: .dockerignore didn't exclude local .env files, so the web build would have silently baked a developer's local API URL into the production image. Added a CI job that boots the real docker-compose.yml on GitHub's runners and smoke-tests login plus a metrics endpoint — the actual proof of a reproducible one-command environment.

Phase 5 — The Directors' Report feature

- The brief's real ask, reframed: not just a metrics dashboard, but a document that saves the Director time on the paperwork of public correspondence. generate_directors_report() assembles a five-section narrative (Overview,

Financial Review, Commercial Progress, Cash Position & Going Concern, Outlook) grounded in the validated data, in the register of a genuine half-year Directors' statement.

- Extended narrative extraction to pull the Senus 2030 strategy targets (50% CAGR, FY2028 EBITDA-positive) out of the filing's prose — cited only when actually extracted for that filing, never as a hardcoded fallback.
- Ran the app's color choices through an automated CVD (colorblindness) validator rather than eyeballing them: the original green/red KPI coloring failed outright on deuteranopia separation — the classic red/green colorblindness collision, on the exact colors marking "good" vs "bad". Replaced with a validated status pair plus a mandatory arrow icon, since color is never allowed to carry meaning alone.

Phase 6 — Role-based access, theme, and polish

- Backend-enforced access control for the four named readers: a disallowed category returns 403, not just a hidden nav link. Management sees everything; the Board sees everything except two operational sales-tracking figures; Equity Investors get full depth on Growth and Returns with balance-sheet detail trimmed to headline figures; Credit Providers get full depth on Cash & Solvency with Returns hidden entirely, since ROCE isn't a lender's concern. Seven new tests cover the policy directly.
- Frontend: role-aware theme (a distinct accent color per role, sharing the same validated data-viz palette), a nav that hides what a role's endpoint would 403 on, and an honest on-page note whenever a view is a trimmed summary rather than full detail.
- Replaced mechanical label derivation ("Ebitda", "Roce", "pnl", "extracted") with hand-authored display names throughout, and gave the app a distinct report identity — serif headings, monospace section labels — instead of a generic dark-SaaS-dashboard look.

The four Board Report readers

The brief names four readers with different concerns. Access is nested by design and enforced on the backend, not just hidden in the UI — a disallowed category 403s if requested directly.

Management	Full detail, every category — Growth, Profitability, Cash & Liquidity, Solvency, Returns.
Board	Everything Management sees, except two operational sales-tracking figures (a day-to-day management concern, not a governance one).
Equity Investor	Full depth on Growth and Returns (their upside case); Cash & Liquidity and Solvency trimmed to headline cash position and net assets only.
Credit Provider	Full depth on Cash & Liquidity and Solvency (repayment risk); Growth and Profitability trimmed to headline figures; Returns (ROCE) hidden entirely — not a lender's concern.

Validation, not assumption

Every extracted figure is cross-checked against its components before being trusted — a row that fails reconciliation is flagged failed_validation and excluded from computed metrics, not silently accepted. This caught two real bugs during development, both now regression-tested: a balance-sheet sign-convention error that made bank debt and working capital compute backwards (liabilities print as negative figures in the source filing), and a reconciliation rule with the wrong sign for the operating-loss check. Both were found by running the code against the filing's actual numbers, not by inspection.

Current status

Backend	30/30 tests passing; extraction, metrics, reconciliation, role access, and the Directors' Report generator all covered.
Frontend	Verified live in-browser across all four role logins — correct theme, correct nav, correct trimmed/hidden data per role.
CI	GitHub Actions runs the test suite, a Next.js production build, and a full docker-compose smoke test on every push.

Deployment	Frontend live on Vercel. API + Postgres are built for Render (render.yaml, Dockerfile committed) but not yet live at submission time — the demo runs against the local API in the meantime.
Repository	github.com/RealRyanno2022/assiduous-technology-graduate-assessment — clean, logically-scoped commit history matching the phases above.

Built with Claude Code (Claude Sonnet 5) as the primary development tool — architecture, backend, frontend, the role-based access system, CI/CD. Every change was verified before being accepted, not just generated: tests run after each backend change, the app driven live in-browser after each frontend change, colors validated automatically rather than eyeballed.